

Azure Databricks Delta Lake Assignment

Phase 1: Dataset Creation and Data Cleaning using Google Colab

Why Google Colab?

Google Colab was used to generate sample customer datasets and perform data cleaning using the Pandas library before processing the data in Azure Databricks.

What I Did

1. Imported Required Libraries

- Imported Pandas and NumPy for data generation and manipulation.

2. Created Customer Master Dataset

- Generated a sample dataset containing customer information such as Customer ID, Name, City, Age, Product, Quantity, Price, and Total Amount.

3. Created Customer Incremental Dataset

- Created a second dataset containing both updated customer records and new customer records to simulate incoming data.

4. Introduced Data Quality Issues

- Added missing values and duplicate records intentionally to simulate real-world data.

5. Performed Data Cleaning

- Checked for missing values.
- Filled missing values using appropriate methods.
- Removed duplicate records.
- Verified data types.
- Recalculated the Total Amount column.

6. Saved Cleaned Datasets

- Exported the cleaned datasets as:
 - customer_master_cleaned.csv
 - customer_incremental_cleaned.csv

Step 1: Create Customer Master Dataset

The Customer Master dataset represents the existing customer records stored in the company database.

```
1 customer_ids = list(range(1001, 1101))
2
3 names = [
4     "Aarav","Vivaan","Aditya","Vihaan","Arjun","Sai","Krishna","Ishaan",
5     "Ananya","Diya","Aadhya","Sara","Riya","Priya","Sneha","Pooja",
6     "Rahul","Rohan","Amit","Kunal","Neha","Meera","Nikhil","Kiran"
7 ]
8
9 cities = [
10    "Pune","Mumbai","Nashik","Nagpur","Delhi",
11    "Bengaluru","Hyderabad","Chennai","Jaipur","Indore"
12 ]
13
14 master_df = pd.DataFrame({
15     "customer_id": customer_ids,
16     "customer_name": np.random.choice(names,100),
17     "city": np.random.choice(cities,100),
18     "age": np.random.randint(20,60,100),
19     "product": np.random.choice(
20         ["Laptop","Phone","Tablet","Headphones","Keyboard","Mouse"],100),
21     "quantity": np.random.randint(1,6,100),
22     "price": np.random.randint(500,50000,100)
23 })
24
25 master_df["total_amount"] = master_df["quantity"] * master_df["price"]
26
27 master_df.head()
```

	customer_id	customer_name	city	age	product	quantity	price	total_amount
0	1001	Krishna	Chennai	38	Phone	5	49549	247745
1	1002	Kunal	Chennai	21	Keyboard	4	15805	63220
2	1003	Sneha	Nashik	45	Mouse	5	10317	51585

Screenshot: Importing Pandas and NumPy libraries

Introduce Data Quality Issues

Real-world data is rarely perfect.

To simulate realistic scenarios, we intentionally introduce:

- Missing values
- Duplicate records

```
1 master_df.loc[5,"city"] = np.nan
2 master_df.loc[15,"customer_name"] = np.nan
3 master_df.loc[25,"price"] = np.nan
4
5 duplicate_rows = master_df.iloc[[10,20]]
6
7 master_df = pd.concat([master_df,duplicate_rows],ignore_index=True)
8
9 master_df
```

	customer_id	customer_name	city	age	product	quantity	price	total_amount
0	1001	Krishna	Chennai	38	Phone	5	49549.0	247745
1	1002	Kunal	Chennai	21	Keyboard	4	15805.0	63220
2	1003	Sneha	Nashik	45	Mouse	5	10317.0	51585
3	1004	Aadhya	Pune	51	Laptop	4	11761.0	47044
4	1005	Ishaan	Chennai	25	Headphones	3	917.0	2751
...	...	...	...	...	...	...	...	...
97	1098	Kiran	Delhi	27	Headphones	1	47232.0	47232
98	1099	Aarav	Pune	46	Headphones	2	39581.0	79162
99	1100	Sara	Pune	46	Headphones	5	28069.0	140345
100	1011	Aadhya	Hyderabad	43	Headphones	3	39013.0	117039
101	1021	Kiran	Chennai	35	Mouse	4	39256.0	157024

Screenshot: Creating the customer master dataset

Generate Incremental Dataset

This dataset simulates newly received customer records.

It contains:

- Existing customers with updated information
- Completely new customers

```
[ ] 1 increment_df = pd.DataFrame({
2     "customer_id": [1003, 1012, 1025, 1040, 1055, 1101, 1102, 1103, 1104, 1105],
3     "customer_name": [
4         "Aditya", "Sneha", "Rahul", "Meera", "Aarav",
5         "Yash", "Tanvi", "Om", "Siya", "Ved"
6     ],
7     "city": {
8         "Pune", "Mumbai", "Delhi", "Nagpur", "Hyderabad",
9         "Surat", "Lucknow", "Pune", "Mumbai", "Goa"
10    },
11    "age": [31, 29, 40, 36, 27, 25, 24, 28, 30, 26],
12    "product": [
13        "Laptop", "Phone", "Tablet", "Keyboard", "Mouse",
14        "Laptop", "Phone", "Tablet", "Laptop", "Mouse"
15    ],
16    "quantity": [2, 1, 3, 1, 4, 2, 1, 2, 3, 1],
17    "price": [60000, 30000, 25000, 4000, 1500, 45000, 22000, 18000, 50000, 2000]
18 })
19
20 increment_df["total_amount"] = increment_df["quantity"] * increment_df["price"]
21
22 increment_df
```

	customer_id	customer_name	city	age	product	quantity	price	total_amount
0	1003	Aditya	Pune	31	Laptop	2	60000	120000
1	1012	Sneha	Mumbai	29	Phone	1	30000	30000
2	1025	Rahul	Delhi	40	Tablet	3	25000	75000

Screenshot: Creating the customer incremental dataset

Save the Datasets

The generated datasets are saved as CSV files and will be uploaded to Azure Databricks for further processing.

```

1 master_df.to_csv("customer_master.csv",index=False)
2
3 increment_df.to_csv("customer_incremental.csv",index=False)
4
5 print("Datasets saved successfully.")

```

Datasets saved successfully.

```

1 print(master_df.shape)
2 print(increment_df.shape)

```

```

(102, 8)
(10, 8)

```

```

1 master_df.sample(10)

```

	customer_id	customer_name	city	age	product	quantity	price	total_amount
100	1011	Aadhya	Hyderabad	43	Headphones	3	39013.0	117039
47	1048	Priya	Hyderabad	45	Tablet	2	1228.0	2456
41	1042	Krishna	Nagpur	42	Phone	4	21852.0	87408
6	1007	Krishna	Nashik	23	Keyboard	5	20858.0	104290
21	1022	Sara	Delhi	48	Tablet	5	19388.0	96940
39	1040	Arjun	Hyderabad	47	Phone	1	47820.0	47820
42	1043	Neha	Nagpur	50	Tablet	5	46852.0	234260
81	1082	Nikhil	Pune	23	Mouse	5	38802.0	193010
1	1002	Kunal	Chennai	21	Keyboard	4	15805.0	63220
98	1099	Aarav	Pune	46	Headphones	2	39581.0	79162

Screenshot: Introducing missing values and duplicate records

▼ Data Cleaning using Pandas

Before using the datasets in Azure Databricks, we clean the data by:

- Exploring the dataset
- Checking data types
- Handling missing values
- Removing duplicate records
- Validating data
- Saving the cleaned datasets

```
[ ] 1 master_df = pd.read_csv("customer_master.csv")
    2 increment_df = pd.read_csv("customer_incremental.csv")
```

+ Code

+ Text

```
[ ] 1 master_df.head()
```

	customer_id	customer_name	city	age	product	quantity	price	total_amount
0	1001	Krishna	Chennai	38	Phone	5	49549.0	247745
1	1002	Kunal	Chennai	21	Keyboard	4	15805.0	63220
2	1003	Sneha	Nashik	45	Mouse	5	10317.0	51585
3	1004	Aadhya	Pune	51	Laptop	4	11781.0	47044
4	1005	Ishaan	Chennai	25	Headphones	3	917.0	2751

```
[ ] 1 master_df.tail()
```

	customer_id	customer_name	city	age	product	quantity	price	total_amount
97	1008	Kiran	Delhi	27	Headphones	1	47232.0	47232
98	1009	Aarav	Pune	40	Headphones	2	39581.0	79162
99	1100	Sara	Pune	40	Headphones	5	28089.0	140345

Screenshot: Checking and handling missing values

```
1 master_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 102 entries, 0 to 101
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   customer_id     102 non-null   int64
1   customer_name   101 non-null   object
2   city            101 non-null   object
3   age             102 non-null   int64
4   product         102 non-null   object
5   quantity        102 non-null   int64
6   price           101 non-null   float64
7   total_amount    102 non-null   int64
dtypes: float64(1), int64(4), object(3)
memory usage: 6.5+ KB
```

+ Code

+ Text

```
1 master_df.describe()
```

	customer_id	age	quantity	price	total_amount
count	102.000000	102.000000	102.000000	101.000000	102.000000
mean	1049.823529	40.000000	3.186275	25147.019802	79877.754902
std	29.130778	11.704277	1.440093	14784.565424	63803.663728
min	1001.000000	20.000000	1.000000	912.000000	2396.000000
25%	1024.250000	30.000000	2.000000	11781.000000	28401.000000
50%	1049.500000	39.500000	3.000000	24076.000000	61650.000000
75%	1074.750000	51.000000	4.750000	38341.000000	119897.500000
max	1100.000000	59.000000	5.000000	49549.000000	247745.000000

Screenshot: Removing duplicates and verifying data types

```
1 master_df.duplicated().sum()
np.int64(0)

2 master_df.dtypes
2
customer_id    int64
customer_name  object
city           object
age           int64
product        object
quantity       int64
price         float64
total_amount   int64
dtype: object

1 master_df["total_amount"] = master_df["price"] * master_df["quantity"]

1 master_df.head()
```

	customer_id	customer_name	city	age	product	quantity	price	total_amount
0	1001	Krishna	Chennai	38	Phone	5	49549.0	247745.0
1	1002	Kunal	Chennai	21	Keyboard	4	15805.0	63220.0
2	1003	Sneha	Nashik	45	Mouse	5	10317.0	51585.0
3	1004	Aadhy	Pune	51	Laptop	4	11761.0	47044.0
4	1005	Ishaan	Chennai	25	Headphones	3	917.0	2751.0

Screenshot: Recalculating the Total Amount column

```
1 master_df["total_amount"] = master_df["price"] * master_df["quantity"]

1 master_df.head()
```

	customer_id	customer_name	city	age	product	quantity	price	total_amount
0	1001	Krishna	Chennai	38	Phone	5	49549.0	247745.0
1	1002	Kunal	Chennai	21	Keyboard	4	15805.0	63220.0
2	1003	Sneha	Nashik	45	Mouse	5	10317.0	51585.0
3	1004	Aadhy	Pune	51	Laptop	4	11761.0	47044.0
4	1005	Ishaan	Chennai	25	Headphones	3	917.0	2751.0

```
1 master_df.shape
(100, 8)

1 master_df.to_csv("customer_master_cleaned.csv", index=False)
2
3 increment_df.to_csv("customer_incremental_cleaned.csv", index=False)

1 from google.colab import files
2
3 files.download("customer_master_cleaned.csv")
4 files.download("customer_incremental_cleaned.csv")
```

Screenshot: Exporting the cleaned CSV files

Why I Did This

- To simulate a real-world ETL process.
- To ensure high-quality data before loading it into Delta Lake.
- To prepare datasets for implementing Delta Lake MERGE operations.

Phase 2: Delta Lake Implementation using Azure Databricks

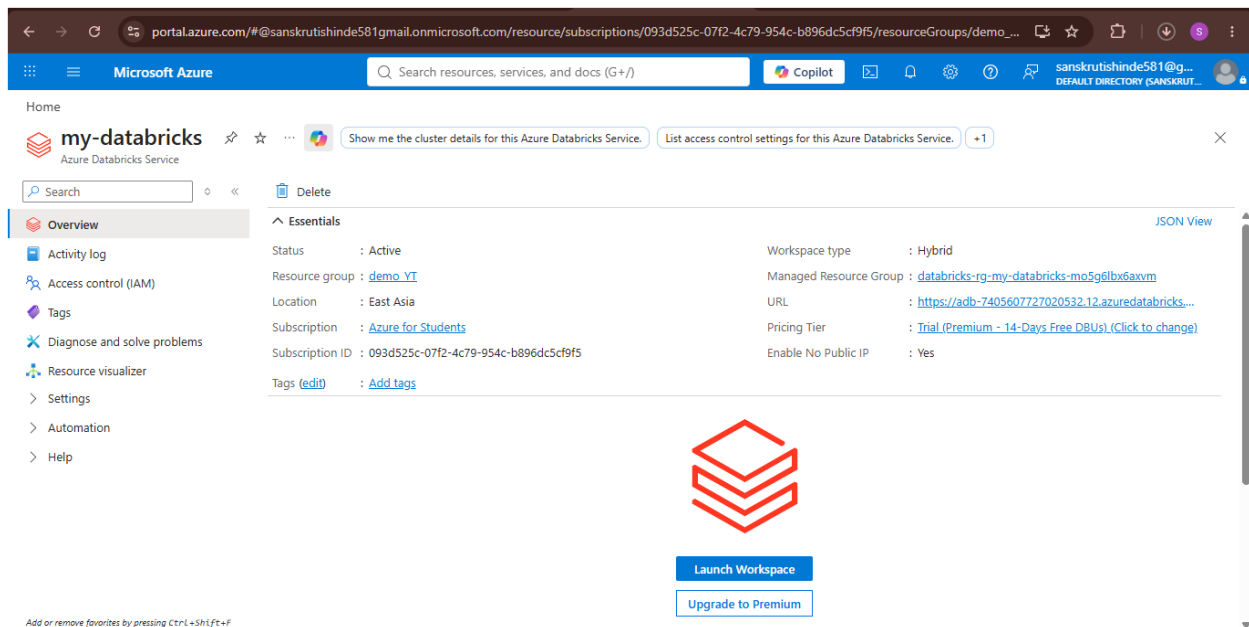
Why Azure Databricks?

Azure Databricks provides Apache Spark and Delta Lake capabilities for processing large datasets, implementing ACID transactions, and performing MERGE operations efficiently.

What I Did

1. Created Azure Databricks Compute

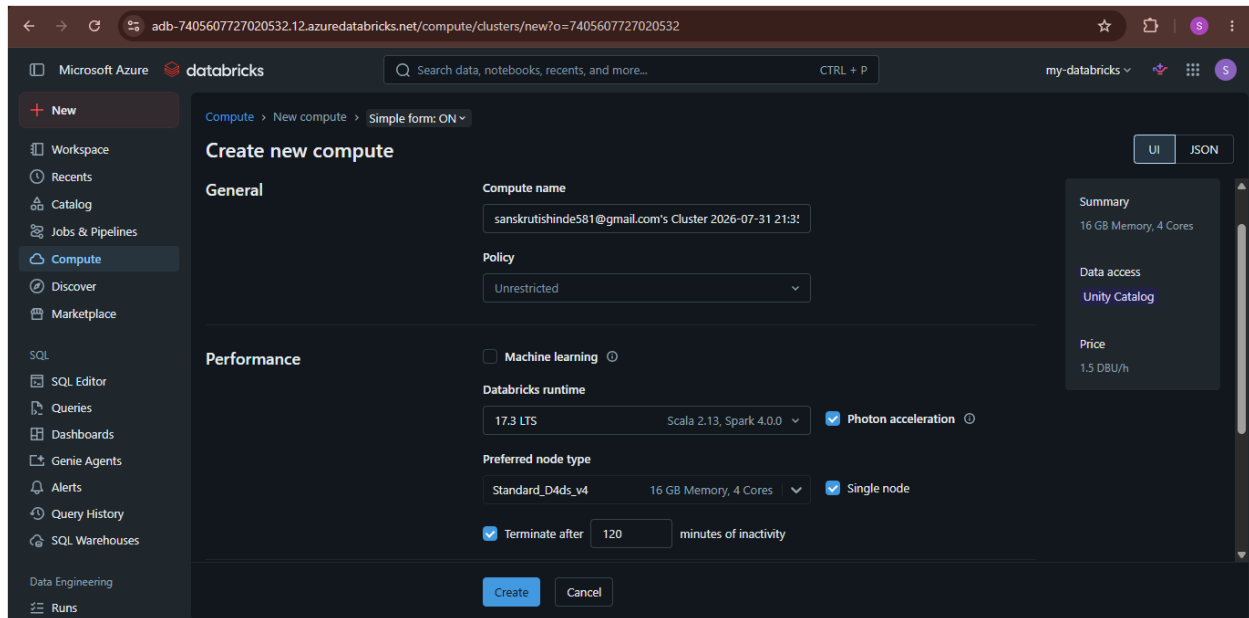
- Created a compute cluster to execute Spark workloads.



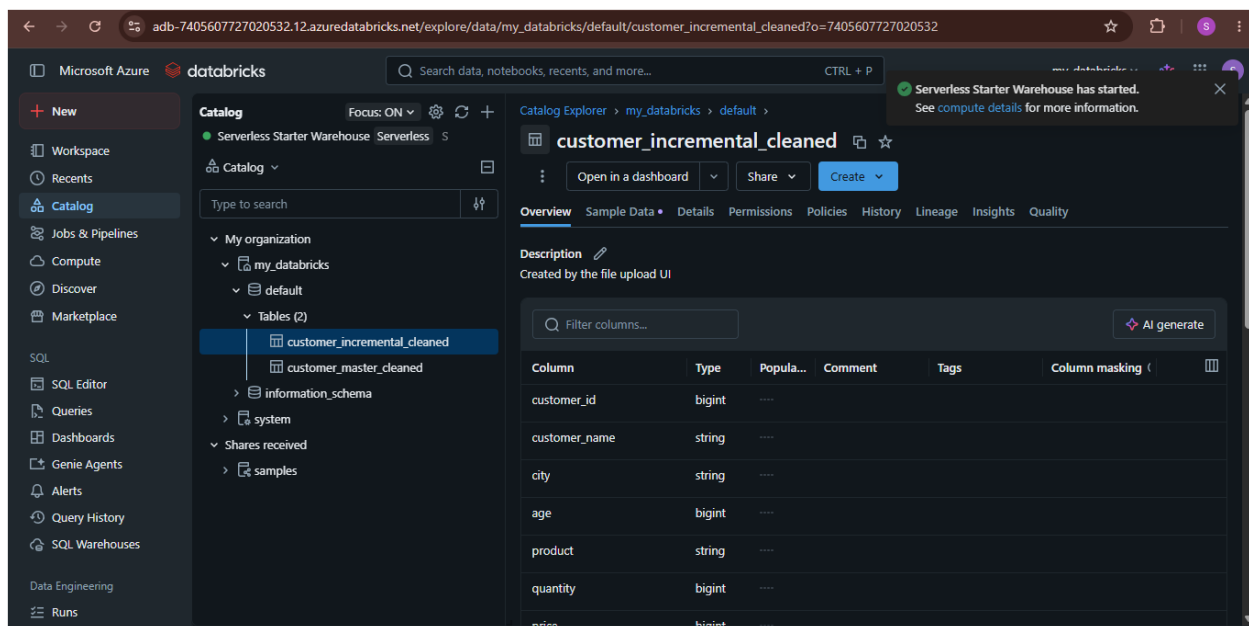
Screenshot 1: Azure Databricks workspace and compute cluster

2. Uploaded Cleaned CSV Files

- Uploaded the cleaned customer datasets into Azure Databricks.
- Created tables from the uploaded CSV files.



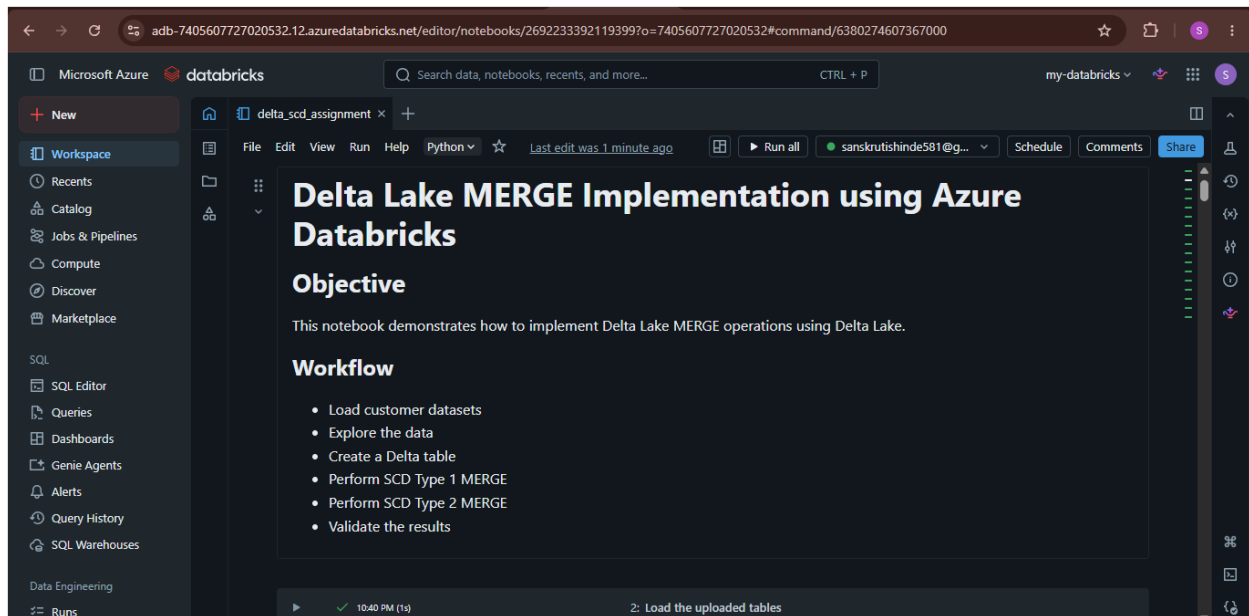
Screenshot 2: Customer master table upload



Screenshot 3: Customer incremental table upload

3. Loaded Data into Spark DataFrames

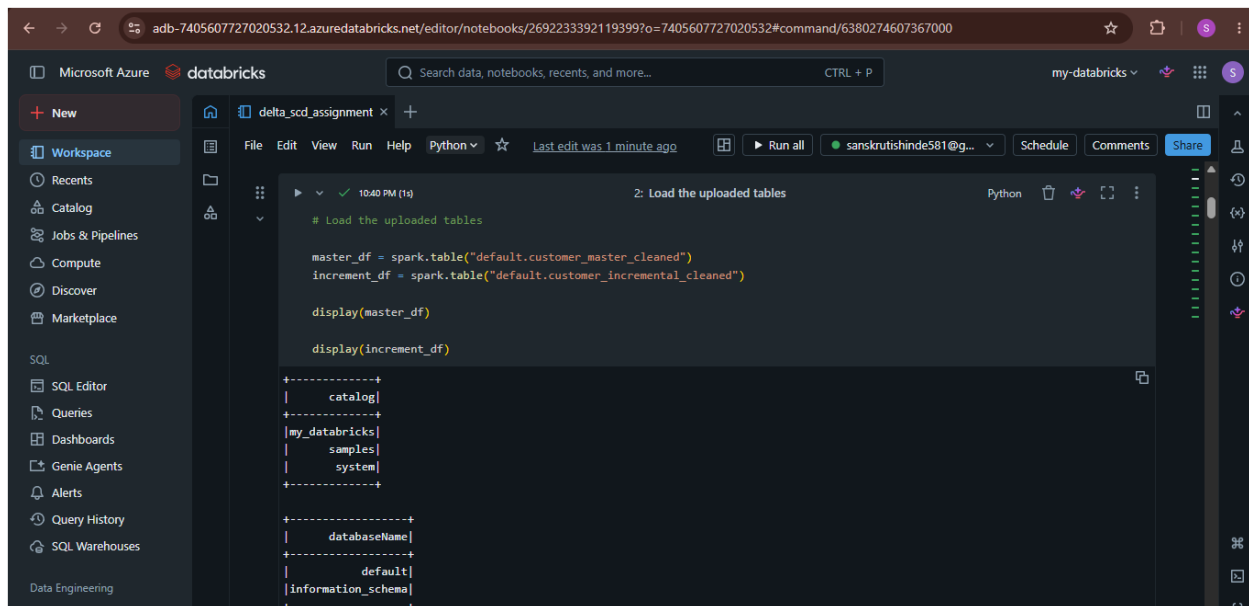
- Loaded the customer master and incremental datasets using Spark.



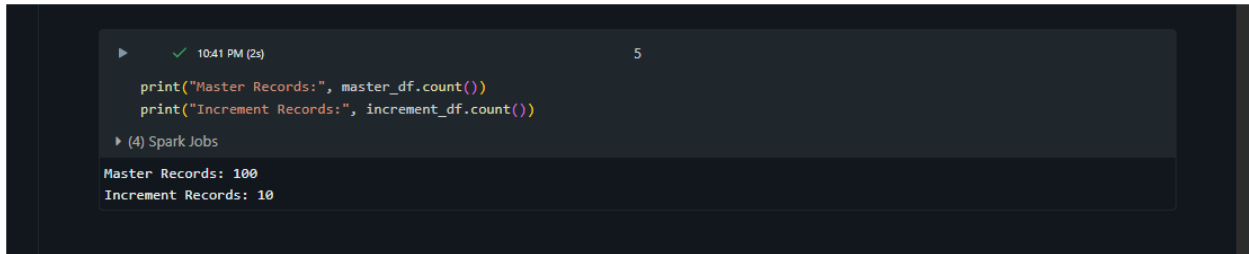
Screenshot 4: Loading customer datasets into Spark DataFrames

4. Verified the Data

- Displayed the datasets.
- Checked schemas and record counts.



Screenshot 5: Dataset verification

A screenshot of a Jupyter Notebook cell. The cell contains two lines of Python code: `print("Master Records:", master_df.count())` and `print("Increment Records:", increment_df.count())`. Below the code, it says "(4) Spark Jobs". The output of the cell shows "Master Records: 100" and "Increment Records: 10".

```
1041 PM (2s) 5
print("Master Records:", master_df.count())
print("Increment Records:", increment_df.count())
(4) Spark Jobs
Master Records: 100
Increment Records: 10
```

Screenshot: Schema and record count check

5. Created a Delta Table

- Converted the customer master dataset into a Delta table for transactional processing.

A screenshot of a Jupyter Notebook showing two cells. The first cell is titled "Step 1: Create Delta Table" and contains a text block explaining that the cleaned master dataset is stored in Delta format. The second cell contains code to write the master dataset as a Delta table. The output of the second cell shows the creation of the Delta table and its display.

Step 1: Create Delta Table

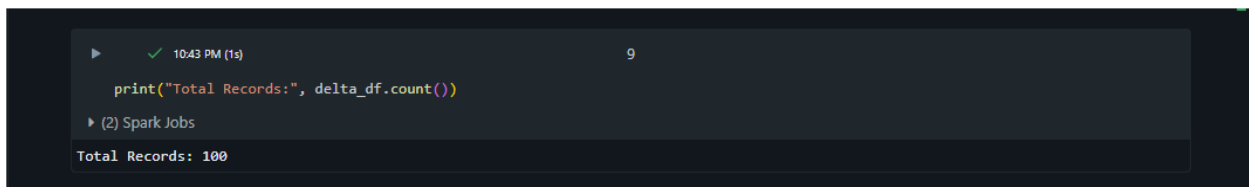
The cleaned master dataset is stored in Delta format. Delta Lake provides ACID transactions, schema enforcement, and supports MERGE operations.

```
1043 PM (3s) 7
# Save the master dataset as a Delta table

master_df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("customer_delta")
(1) Spark Jobs

1043 PM (1s) 8
delta_df = spark.table("customer_delta")
display(delta_df)
(1) Spark Jobs
```

Screenshot: Creating the Delta table

A screenshot of a Jupyter Notebook cell. The cell contains one line of Python code: `print("Total Records:", delta_df.count())`. Below the code, it says "(2) Spark Jobs". The output of the cell shows "Total Records: 100".

```
1043 PM (1s) 9
print("Total Records:", delta_df.count())
(2) Spark Jobs
Total Records: 100
```

Screenshot: Delta table creation confirmation

Why I Created a Delta Table

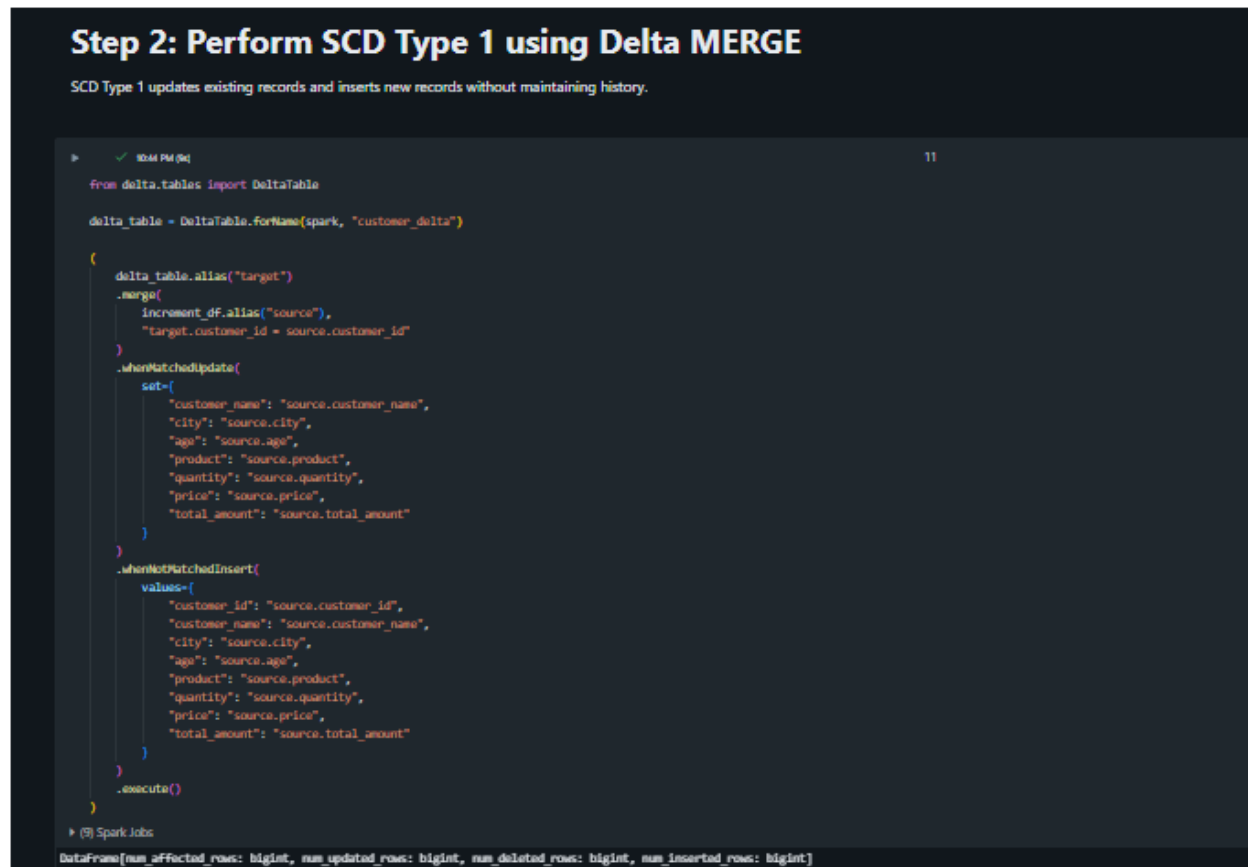
- To store data in Delta Lake format.
- To support ACID transactions.
- To enable efficient MERGE operations.

- To improve data reliability and consistency.

SCD Type 1 Implementation

What I Did

- Performed a Delta Lake MERGE operation.
- Updated existing customer records using the incremental dataset.
- Inserted new customer records that did not exist in the master table.



```
Step 2: Perform SCD Type 1 using Delta MERGE
SCD Type 1 updates existing records and inserts new records without maintaining history.

from delta.tables import DeltaTable

delta_table = DeltaTable.forName(spark, "customer_delta")

(
  delta_table.alias("target")
    .merge(
      increment_df.alias("source"),
      "target.customer_id = source.customer_id"
    )
    .whenMatchedUpdate(
      set={
        "customer_name": "source.customer_name",
        "city": "source.city",
        "age": "source.age",
        "product": "source.product",
        "quantity": "source.quantity",
        "price": "source.price",
        "total_amount": "source.total_amount"
      }
    )
    .whenNotMatchedInsert(
      values={
        "customer_id": "source.customer_id",
        "customer_name": "source.customer_name",
        "city": "source.city",
        "age": "source.age",
        "product": "source.product",
        "quantity": "source.quantity",
        "price": "source.price",
        "total_amount": "source.total_amount"
      }
    )
    .execute()
)

DataFrame[num_affected_rows: bigint, num_updated_rows: bigint, num_deleted_rows: bigint, num_inserted_rows: bigint]
```

Screenshot: Delta Lake MERGE operation for SCD Type 1

10:44 PM (1s) 12

```
display(spark.table("customer_delta"))
```

(2) Spark Jobs

	customer_id	customer_name	city	age	product	quantity	price	total_amount
1	1001	Krishna	Chennai	38	Phone	5	49549	
2	1002	Kunal	Chennai	21	Keyboard	4	15805	
3	1004	Aadhya	Pune	51	Laptop	4	11761	
4	1005	Ishaan	Chennai	25	Headphones	3	917	
5	1006	Neha	Unknown	51	Laptop	4	42076	
6	1007	Krishna	Nashik	23	Keyboard	5	20858	
7	1008	Amit	Pune	30	Headphones	2	3767	
8	1009	Nikhil	Delhi	36	Keyboard	4	17709	
9	1010	Aadhya	Indore	57	Tablet	2	24552	
10	1011	Aadhya	Hyderabad	43	Headphones	3	39013	
11	1013	Neha	Jaipur	53	Laptop	3	11838	

Screenshot: Updated and inserted records after MERGE

10:45 PM (1s) 13

```
print("Records after MERGE:", spark.table("customer_delta").count())
```

(2) Spark Jobs

Records after MERGE: 105

Screenshot: MERGE operation confirmation

Why I Did It

SCD Type 1 is used when historical data does not need to be preserved. Existing records are simply overwritten with the latest values.

Result

- Initial Records: 100
- Updated Existing Records: 5
- Newly Inserted Records: 5
- Final Records: 105

SCD Type 2 Implementation

What I Did

- Created a new Delta table with history-tracking columns:
- is_current
- start_date

- end_date

Step 3: Implement SCD Type 2

SCD Type 2 preserves historical data by creating a new version of a record whenever changes occur.

Additional columns used:

- is_current
- start_date
- end_date

```
from pyspark.sql.functions import lit, current_date

# Create SCD2 table from current Delta table
scd2_df = spark.table("customer_delta") \
    .withColumn("is_current", lit(True)) \
    .withColumn("start_date", current_date()) \
    .withColumn("end_date", lit(None).cast("date"))

scd2_df.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("customer_delta_scd2")

display(spark.table("customer_delta_scd2"))
```

(4) Spark Jobs

scd2_df: pyspark.sql.connect.dataframe.DataFrame = [customer_id: long, customer_name: string ... 9 more fields]

	customer_id	customer_name	city	age	product	quantity	price	total_amount	is_current	start_date	end_date
1	1001	Krishna	Chennai	38	Phone	5	49549	247745	true	2026-07-31	null
2	1002	Kunal	Chennai	21	Keyboard	4	15805	63220	true	2026-07-31	null
3	1004	Radhika	Bangalore	51	Laptop	4	11761	47044	true	2026-07-31	null

Screenshot: Creating the SCD Type 2 Delta table

```
spark.table("customer_delta_scd2").printSchema()
```

```
root
 |-- customer_id: long (nullable = true)
 |-- customer_name: string (nullable = true)
 |-- city: string (nullable = true)
 |-- age: long (nullable = true)
 |-- product: string (nullable = true)
 |-- quantity: long (nullable = true)
 |-- price: double (nullable = true)
 |-- total_amount: double (nullable = true)
 |-- is_current: boolean (nullable = true)
 |-- start_date: date (nullable = true)
 |-- end_date: date (nullable = true)
```

Screenshot: History-tracking columns in the SCD Type 2 table

Step 4: Validation

Validate the Delta tables after implementing SCD Type 1 and SCD Type 2.

```
print("Customer Delta Records:", spark.table("customer_delta").count())
print("Customer Delta SCD2 Records:", spark.table("customer_delta_scd2").count())
```

(4) Spark Jobs

Customer Delta Records: 105
Customer Delta SCD2 Records: 105

Screenshot: SCD Type 2 table output

Why I Did It

SCD Type 2 is used to maintain historical versions of records. It helps track changes over time by identifying current and historical records.

Validation

What I Did

- Verified record counts.
- Displayed the Delta tables.
- Executed SQL queries to validate the results.
- Confirmed successful implementation of the MERGE operation.



The screenshot shows a SQL IDE interface. At the top, there's a status bar with a green checkmark, the time '10:47 PM (2s)', and the page number '19'. Below this, a SQL query is entered in a dark-themed editor:

```
%sql
SELECT *
FROM customer_delta
LIMIT 10;
```

Below the query editor, it says '(1) Spark Jobs' and shows a command: `_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [customer_id: long, customer_name: string ... 6 more fields]`. At the bottom, a table of results is displayed with 10 rows and 9 columns. The columns are: `customer_id`, `customer_name`, `city`, `age`, `product`, `quantity`, `price`, and `total_amount`. The table data is as follows:

	<code>customer_id</code>	<code>customer_name</code>	<code>city</code>	<code>age</code>	<code>product</code>	<code>quantity</code>	<code>price</code>	<code>total_amount</code>
1	1001	Krishna	Chennai	38	Phone	5	49549	247745
2	1002	Kunal	Chennai	21	Keyboard	4	15805	63220
3	1004	Aadhya	Pune	51	Laptop	4	11761	47044
4	1005	Ishaan	Chennai	25	Headphones	3	917	2751
5	1006	Neha	Unknown	51	Laptop	4	42076	168304
6	1007	Krishna	Nashik	23	Keyboard	5	20858	104290
7	1008	Amit	Pune	30	Headphones	2	3767	7534
8	1009	Nikhil	Delhi	36	Keyboard	4	17709	70836
9	1010	Aadhya	Indore	57	Tablet	2	24552	49104
10	1011	Aadhya	Hyderabad	43	Headphones	3	39013	117039

Screenshot: Verifying record counts

10:47 PM (2s) 20

```
%sql
SELECT *
FROM customer_delta_scd2
WHERE is_current = true;
```

(2) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [customer_id: long, customer_name: string ... 9 more fields]

	customer_id	customer_name	city	age	product	quantity	price	total_amount	is_current
1	1001	Krishna	Chennai	38	Phone	5	49549	247745	true
2	1002	Kunal	Chennai	21	Keyboard	4	15805	63220	true
3	1004	Aadhya	Pune	51	Laptop	4	11761	47044	true
4	1005	Ishaan	Chennai	25	Headphones	3	917	2751	true
5	1006	Neha	Unknown	51	Laptop	4	42076	168304	true
6	1007	Krishna	Nashik	23	Keyboard	5	20858	104290	true
7	1008	Amit	Pune	30	Headphones	2	3767	7534	true
8	1009	Nikhil	Delhi	36	Keyboard	4	17709	70836	true
9	1010	Aadhya	Indore	57	Tablet	2	24552	49104	true
10	1011	Aadhya	Hyderabad	43	Headphones	3	39013	117039	true
11	1013	Neha	Jaipur	53	Laptop	3	11838	35514	true

Screenshot: SQL query validation

10:47 PM (1s) 21

```
display(
  spark.table("customer_delta_scd2")
    .filter("is_current = true")
)
```

(2) Spark Jobs

	customer_id	customer_name	city	age	product	quantity	price	total_amount	is_current
1	1001	Krishna	Chennai	38	Phone	5	49549	247745	true
2	1002	Kunal	Chennai	21	Keyboard	4	15805	63220	true
3	1004	Aadhya	Pune	51	Laptop	4	11761	47044	true
4	1005	Ishaan	Chennai	25	Headphones	3	917	2751	true
5	1006	Neha	Unknown	51	Laptop	4	42076	168304	true
6	1007	Krishna	Nashik	23	Keyboard	5	20858	104290	true
7	1008	Amit	Pune	30	Headphones	2	3767	7534	true
8	1009	Nikhil	Delhi	36	Keyboard	4	17709	70836	true
9	1010	Aadhya	Indore	57	Tablet	2	24552	49104	true
10	1011	Aadhya	Hyderabad	43	Headphones	3	39013	117039	true
11	1013	Neha	Jaipur	53	Laptop	3	11838	35514	true

Screenshot: Final Delta table results

Why I Did It

Validation ensures that:

- Data has been loaded correctly.
- MERGE operations completed successfully.
- Record counts are accurate.
- The final Delta tables contain the expected data.

Conclusion

This assignment successfully demonstrated the complete Delta Lake workflow using Google Colab and Azure Databricks. Initially, I generated and cleaned customer datasets using Pandas in Google Colab. I then uploaded the cleaned datasets to Azure Databricks, where I created Delta tables and implemented Slowly Changing Dimension (SCD) Type 1 using the MERGE operation. I also created an SCD Type 2 table with history-tracking columns, and validated the results using Spark and SQL queries. This workflow illustrates a typical ETL pipeline, combining data preparation, transformation, and Delta Lake operations for reliable and efficient data management.